

Department of Computer Science & Engineering

Microprocessor & Computer Architecture

UE24CS251B

MPCA Lab Programs – 4

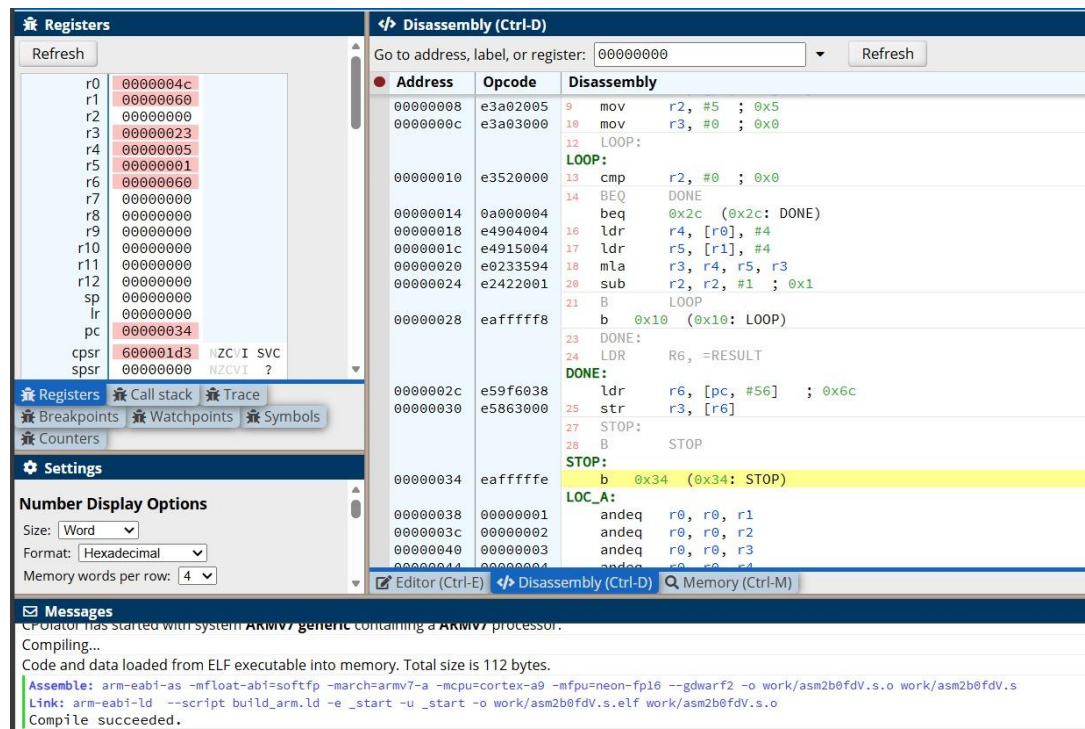
NAME: DHRITI KAMANI

SRN: PES1UG24AM085

CLASS: 4B

- Write an ALP to perform Convolution using MLA instruction (Addition of multiplication of respective numbers of loc A and loc B).

Answer:



Registers

Register	Value	Comment
r0	0000004c	
r1	00000060	
r2	00000000	
r3	00000023	
r4	00000005	
r5	00000001	
r6	00000060	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000034	
cpsr	600001d3	NZC I SVC
spsr	00000000	NZCVI ?

Disassembly (Ctrl-D)

Address	Opcode	Disassembly
00000008	e3a02005	9 mov r2, #5 ; 0x5
0000000c	e3a03000	10 mov r3, #0 ; 0x0
00000010	e3520000	12 LOOP: cmp r2, #0 ; 0x0
00000014	0a000004	14 BEQ DONE
00000018	e4904004	16 beq 0x2c (0x2c: DONE)
0000001c	e4915004	17 ldr r4, [r0], #4
00000020	e0233594	18 ldr r5, [r1], #4
00000024	e2422001	19 mla r3, r4, r5, r3
00000028	ea000000	20 sub r2, r2, #1 ; 0x1
0000002c	eb000000	21 B LOOP
00000030	eb000000	22 b 0x10 (0x10: LOOP)
00000034	ea000000	23 DONE: LDR R6, =RESULT
00000038	e59f6038	24 DONE: ldr r6, [pc, #56] ; 0x6c
0000003c	e5863000	25 str r3, [r6]
00000040	ea000000	26 STOP: B STOP
00000044	ea000000	27 STOP: b 0x34 (0x34: STOP)
00000048	00000001	28 LOC_A: andeq r0, r0, r1
0000004c	00000002	29 andeq r0, r0, r2
00000050	00000003	30 andeq r0, r0, r3
00000054	00000004	31 andeq r0, r0, r4

Messages

CP-0101 has started with system ARMv7-generic containing a ARMv7 processor.

Compiling...

Code and data loaded from ELF executable into memory. Total size is 112 bytes.

Assemble: arm-eabi-as -mfloat-abi=softfp -march=armv7-a -mcpu=cortex-a9 -mcpu=neon-fp16 --gdwarf2 -o work/asm2b0fdv.s.o work/asm2b0fdv.s

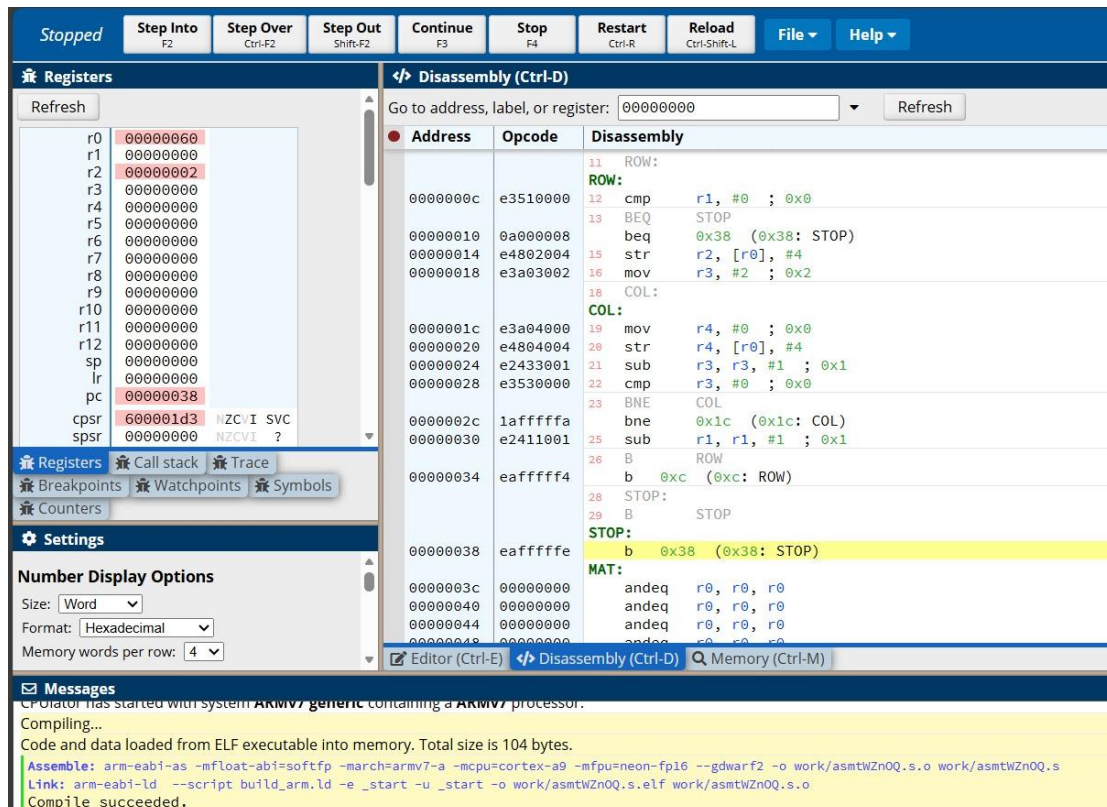
Link: arm-eabi-ld --script build_arm.ld -e _start -u _start -o work/asm2b0fdv.s.elf work/asm2b0fdv.s.o

Compile succeeded.

```
. global _start
_start:
    LDR    R0, =LOC_A
    LDR    R1, =LOC_B
    MOV    R2, #5
    MOV    R3, #0
LOOP:
    CMP    R2, #0
    BEQ    DONE
    LDR    R4, [R0], #4
    LDR    R5, [R1], #4
    MLA    R3, R4, R5, R3
    SUB    R2, R2, #1
    B      LOOP
```

DONE:
 LDR R6, =RESULT
 STR R3, [R6]
 STOP:
 B STOP
 LOC_A:
 .word 1, 2, 3, 4, 5
 LOC_B:
 .word 5, 4, 3, 2, 1
 RESULT:
 .word 0

- 2 **Write a program in ARM7TDMI-ISA to generate a diagonal matrix.**
;Note: do not read the matrix elements.
LET THE DIAGONAL VALUE BE 2



The screenshot shows the ARM7TDMI-ISA simulator interface. The top bar includes buttons for 'Step Into', 'Step Over', 'Step Out', 'Continue', 'Stop', 'Restart', and 'Reload'. Below this is a 'Registers' panel on the left and a 'Disassembly (Ctrl-D)' panel on the right. The 'Registers' panel shows the state of various registers, including r0 through r15, sp, lr, cpsr, and spsr. The 'Disassembly' panel shows the assembly code being executed, with addresses, opcodes, and disassembled instructions. The messages panel at the bottom shows the compilation process, including the assembly and linking steps, and a successful compile message.

```
.global _start
_start:
    LDR R0, =MAT
    MOV R1, #3
    MOV R2, #2
ROW:
    CMP R1, #0
    BEQ STOP
    STR R2, [R0], #4
    MOV R3, #2
COL:
    MOV R4, #0
    STR R4, [R0], #4
```

```

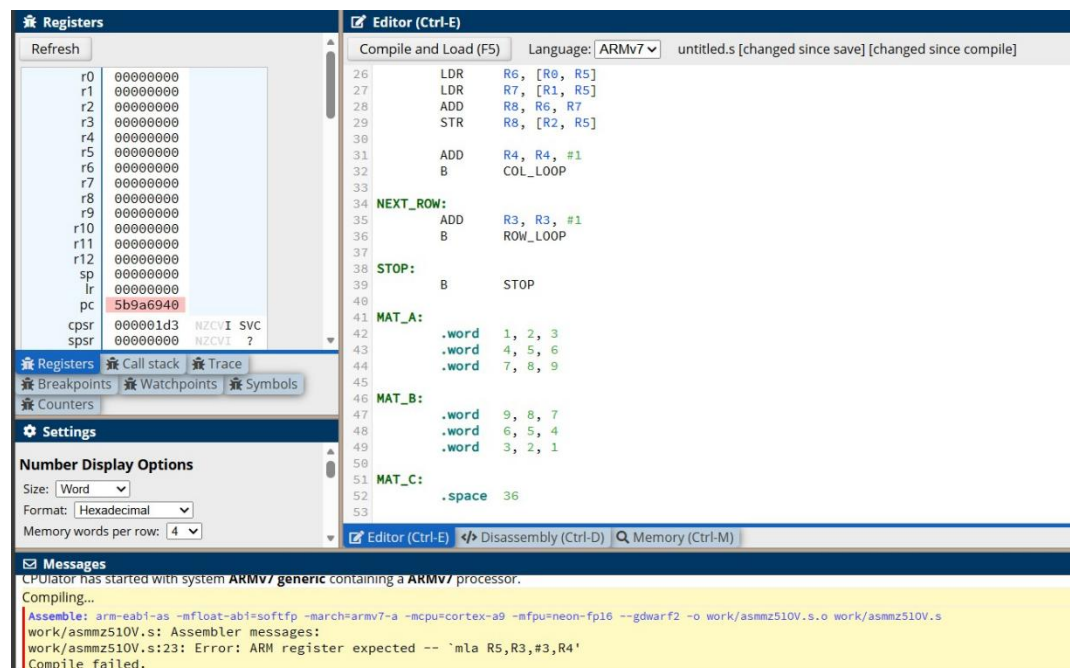
SUB    R3, R3, #1
CMP    R3, #0
BNE    COL
SUB    R1, R1, #1
B      ROW
STOP:
B      STOP
MAT:
.space 36

```

3 Write an ARM ALP to perform element-wise addition of two 3×3 matrices using indexed addressing and nested loops.

Requirements

- Matrices A and B each contain 9 elements (3 rows × 3 columns)
- Matrix C should store the sum of corresponding elements
- Use nested loops to traverse rows and columns
- Use the MLA (Multiply Accumulate) instruction to compute the memory offset
- Each matrix element is a 32-bit word



```

.global _start
_start:
    LDR    R0, =MAT_A
    LDR    R1, =MAT_B
    LDR    R2, =MAT_C
    MOV    R3, #0        @ row index i
ROW_LOOP:
    CMP    R3, #3
    BEQ    STOP
    MOV    R4, #0        @ column index j
COL_LOOP:
    CMP    R4, #3
    BEQ    NEXT_ROW
    MLA    R5, R3, #3, R4 @ R5 = (i * 3) + j
    LSL    R5, R5, #2    @ R5 = offset in bytes
    LDR    R6, [R0, R5]

```

```
LDR    R7, [R1, R5]
ADD    R8, R6, R7
STR    R8, [R2, R5]
```

```
ADD    R4, R4, #1
B      COL_LOOP
NEXT_ROW:
ADD    R3, R3, #1
B      ROW_LOOP
STOP:
B      STOP
```

```
MAT_A:
.word 1, 2, 3
.word 4, 5, 6
.word 7, 8, 9
```

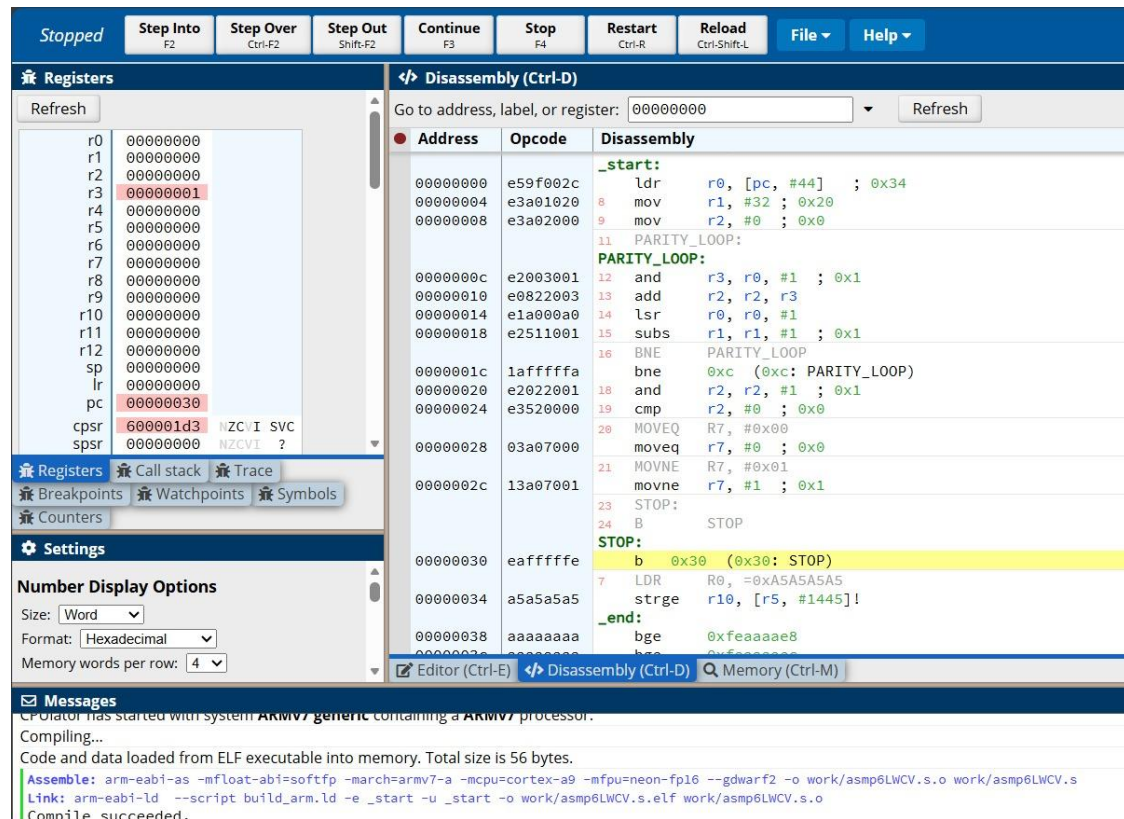
```
MAT_B:
.word 9, 8, 7
.word 6, 5, 4
.word 3, 2, 1
```

```
MAT_C:
.space 36
```

4 ASSIGNMENT QUESTION 1

Write an ARM7TDMI ALP to check whether a given 32-bit number has odd or even parity (number of 1s).

It stores 00 in R7 if parity is EVEN and 01 in R7 if parity is ODD



The screenshot shows the ARM7TDMI ALP simulator interface. The main window displays the disassembly of the program. The program starts at address 00000000 and ends at 00000038. The disassembly shows the following instructions:

```

00000000 e59f002c ldr r0, [pc, #44] ; 0x34
00000004 e3a01020 mov r1, #32 ; 0x20
00000008 e3a02000 mov r2, #0 ; 0x0
0000000c e2003001 and r3, r0, #1 ; 0x1
00000010 e0822003 add r2, r2, r3
00000014 e1a000a0 lsr r0, r0, #1
00000018 e2511001 subs r1, r1, #1 ; 0x1
0000001c 1affffff bne 0xc (0xc: PARITY_LOOP)
00000020 e2022001 and r2, r2, #1 ; 0x1
00000024 e3520000 cmp r2, #0 ; 0x0
00000028 03a07000 moveq r7, #0 ; 0x0
0000002c 13a07001 movne r7, #1 ; 0x1
00000030 eaffffff b 0x30 (0x30: STOP)
00000034 a5a5a5a5 strge r10, [r5, #1445]!
00000038 aaaaaaaa bge 0xfeaaaae8

```

The program also includes a global _start label and a start label.

```

.global _start
start:

```

```
LDR    R0, =0xA5A5A5A5
MOV    R1, #32
MOV    R2, #0
```

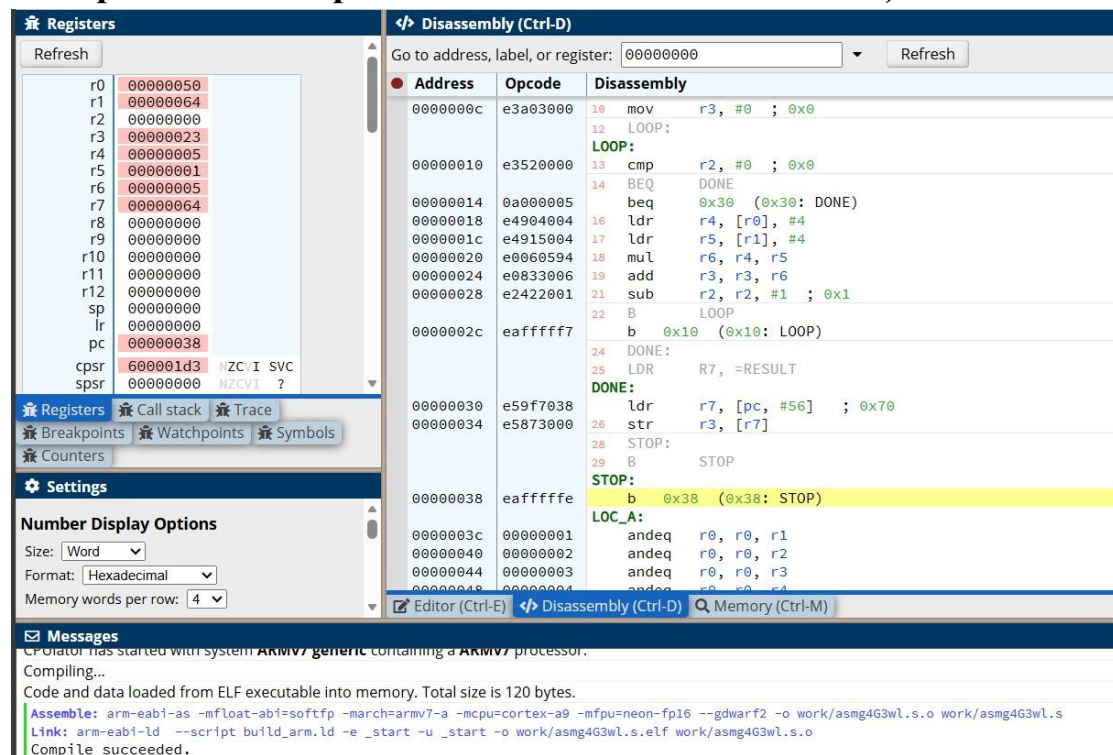
PARITY_LOOP:

```
AND    R3, R0, #1
ADD    R2, R2, R3
LSR    R0, R0, #1
SUBS   R1, R1, #1
BNE    PARITY_LOOP
AND    R2, R2, #1
CMP    R2, #0
MOVEQ  R7, #0x00
MOVNE  R7, #0x01
```

STOP:

```
B      STOP
```

- 5 Write an ALP to perform Convolution using MUL instruction (Addition of multiplication of respective numbers of loc A and loc B)



The screenshot shows the ARMv7 Disassembler interface. The left pane displays the registers (r0-r15, sp, lr, pc, cpsr, spsr) and their current values. The right pane shows the disassembly of the code, with instructions like `mov r3, #0 ; 0x0`, `cmp r2, #0 ; 0x0`, `beq 0x30 (0x30: DONE)`, `ldr r4, [r0], #4`, `ldr r5, [r1], #4`, `mul r6, r4, r5`, `add r3, r3, r6`, `sub r2, r2, #1 ; 0x1`, `b 0x10 (0x10: LOOP)`, `done: ldr r7, =RESULT`, `str r3, [r7]`, `stop: b stop`, and `stop: b 0x38 (0x38: STOP)`. The bottom pane shows the messages, including the compilation and linking process.

```
.global _start
_start:
    LDR    R0, =LOC_A
    LDR    R1, =LOC_B
    MOV    R2, #5
    MOV    R3, #0
LOOP:
    CMP    R2, #0
    BEQ    DONE
    LDR    R4, [R0], #4
    LDR    R5, [R1], #4
    MUL    R6, R4, R5
    ADD    R3, R3, R6
```

	<pre>SUB R2, R2, #1 B LOOP DONE: LDR R7, =RESULT STR R3, [R7] STOP: B STOP LOC_A: .word 1, 2, 3, 4, 5 LOC_B: .word 5, 4, 3, 2, 1 RESULT: .word 0</pre>
--	---